TALLER DE WEBSOCKETS · PROGRAMACIÓN ORIENTADA A LA WEB

Transmisión y difusión de resultados en una jornada electoral

Tres audiencias, ocho servidores sin sesiones y un fan-out de cientos de miles de clientes sobre un backplane en Redis.

Ingeniería de Software · Séptimo semestre · Grupos de máximo tres personas

1. Contexto

Una empresa de tecnología es contratada para operar el **preconteo** de una jornada electoral departamental. Conviene aclarar desde el principio qué es y qué no es ese sistema: el conteo oficial es manual y se hace sobre las actas físicas durante los días siguientes. El preconteo es un reporte informativo cuyo único valor es la **velocidad**, y que vive o muere por su capacidad de mostrar, minuto a minuto, cuántas mesas han reportado y qué dicen. Un preconteo lento no sirve para nada; un preconteo que se cae en el pico es un escándalo; y un preconteo que muestra un número que luego cambia sin explicación destruye la confianza en el operador.

El departamento tiene **1.842 puestos de votación** y **6.400 mesas**. Cuando cierran las urnas, los jurados cuentan los votos, llenan el acta y la transmiten desde una aplicación web: fotografían el acta y digitan los totales por candidato. Toda esa transmisión se concentra en una ventana de **dos horas**, con un pico muy marcado en los primeros cuarenta minutos, cuando cierran casi todas las mesas urbanas al mismo tiempo.

Del otro lado hay una audiencia enorme y nerviosa: la página pública de resultados llegó, en la jornada anterior, a **400.000 navegadores conectados de forma simultánea**, con picos de reconexión masiva cada vez que la red móvil de la ciudad tuvo un bache. Entre esos dos extremos trabaja el equipo de verificación: ciento veinte auditores que revisan las actas que el sistema marca como dudosas, cada una asignada a un solo auditor, desde consolas que deben verse entre sí.

> **El problema real que se debe resolver**
> Un mismo hecho (entra un acta) tiene que llegar, con garantías distintas, a tres audiencias de naturaleza completamente distinta, repartidas sobre varios servidores que no comparten memoria. El caso evalúa tres cosas: si usted elige el transporte adecuado para cada audiencia y lo puede defender con mediciones; si su fan-out sobrevive al pico sin bloquear el Event Loop; y si su sistema distribuido entrega un resultado consistente y explicable cuando las cosas fallan, que es cuando se nota la diferencia entre un prototipo y un sistema.

2. Las tres audiencias

Audiencia	Volumen	Naturaleza del tráfico	Garantía que exige
Jurados y digitadores	6.400 clientes, conectados de forma intermitente desde redes móviles pobres	Bidireccional: envían el acta, reciben confirmación, corrección o rechazo	Que el acta llegue una sola vez y que el jurado sepa con certeza si quedó registrada

Audiencia	Volumen	Naturaleza del tráfico	Garantía que exige
Consolas de verificación	120 auditores, conectados toda la jornada	Bidireccional y colaborativo: ven la cola común, toman casos, se ven entre sí	Que dos auditores jamás trabajen sobre la misma acta y que un auditor que se cae libere su caso
Público	hasta 400.000 navegadores, con reconexiones masivas	Una sola dirección: solo reciben actualizaciones del acumulado	Que el número nunca retroceda sin explicación y que un cliente que se reconecta vea exactamente lo mismo que uno que nunca se desconectó

Las tres audiencias no se sirven con el mismo mecanismo. Parte de la evaluación consiste en que usted elija entre **WebSocket**, **Server-Sent Events** y **polling** para cada una, lo implemente y lo **justifique con mediciones propias**, no con una opinión. La sección 7 fija qué debe medir.

3. Datos y volumen

3.1 Mensaje de transmisión de acta

```
{ "acta": "E14-176-0042-03",        // puesto-mesa-corporacion
  "mesa": 4201, "puesto": 176, "municipio": 11,
  "version": 1,                     // 2, 3... si el acta se corrige
  "jurado": "JUR-88134",
  "ts_cierre": 1793577600,
  "votos": { "C1": 184, "C2": 97, "C3": 311, "BLANCO": 22,
             "NULO": 9, "NO_MARCADO": 4 },
  "sufragantes": 627,
  "hash_foto": "9f2c...",
  "idem": "8f14e45fceea167a"          // clave de idempotencia del envio
}
Regla contable: suma de votos + nulos + no marcados = sufragantes.
Si no cuadra, el acta es dudosa y pasa a verificacion humana.
```

3.2 Volumen y perfil de la jornada

Magnitud	Valor	Implicación
Actas totales	6.400 (más correcciones)	El flujo de escritura es pequeño; el de lectura es gigantesco
Ventana de transmisión	2 horas, con el 55% en los primeros 40 minutos	Pico de cerca de 90 actas por minuto, con ráfagas de 300
Clientes públicos simultáneos	hasta 400.000	Repartidos entre las instancias: cerca de 50.000 por servidor
Instancias de aplicación	8 detrás de un balanceador	Ninguna conoce las conexiones de las demás
Reenvíos del mismo acta	entre 4% y 11%	Redes malas: el jurado reintenta porque no vio la confirmación
Actas corregidas	cerca de 180 en la jornada	Obligan a revertir un valor ya publicado y a explicar el cambio
Actas dudosas	entre 6% y 9%	Van a la cola de verificación humana

4. Requerimiento

Construya el sistema completo, desplegado en la nube, con al menos **ocho instancias de aplicación** detrás de un balanceador de carga y **Redis** como columna vertebral de la comunicación entre ellas. El sistema debe cumplir los siguientes requerimientos funcionales.

RF-1. Recepción de actas con idempotencia

Un acta reenviada cinco veces por un jurado con mala señal debe quedar registrada **una sola vez** y sumar sus votos una sola vez al acumulado. El jurado debe recibir siempre una respuesta inequívoca, incluso para el reenvío: no basta con ignorar el duplicado en silencio. Debe explicar qué ocurre si el servidor se cae exactamente entre el momento en que registra el acta y el momento en que confirma al jurado.

RF-2. Correcciones y convergencia

Un acta puede corregirse: llega la versión 2 con valores distintos y el acumulado debe revertir la versión anterior y aplicar la nueva. Las versiones pueden llegar **fuera de orden** por culpa de los reintentos, de modo que si la versión 3 ya se aplicó, la llegada tardía de la versión 2 no puede retroceder el estado. El resultado final debe depender únicamente del conjunto de actas recibidas, nunca del orden en que llegaron.

RF-3. Acumulados jerárquicos en tiempo real

Cada acta modifica cuatro niveles de acumulado: mesa, puesto, municipio y departamento, además del porcentaje de mesas informadas en cada nivel. Esa actualización debe ser **atómica**: en ningún instante un cliente puede leer un acumulado municipal que ya incluye un acta cuyo detalle de puesto todavía no aparece. Debe explicar cómo garantiza esa atomicidad con Redis y por qué no basta con ejecutar varios comandos seguidos desde la aplicación.

RF-4. Fan-out sin bloquear el Event Loop

Cuando entra un acta, el cambio debe llegar a los clientes públicos interesados. La implementación ingenua recorre el arreglo de 50.000 conexiones de la instancia y llama al envío en cada una: eso bloquea el Event Loop de Node y congela, durante ese tiempo, a todos los demás usuarios del servidor, incluidos los jurados que están transmitiendo. Su solución debe repartir el envío en lotes, ceder el control al Event Loop entre lotes y **medir el retraso del Event Loop** durante el pico para demostrar que se mantiene acotado.

> **La trampa de escala que debe resolver**
> Son 6.400 actas y hasta 400.000 clientes. Si cada acta produce un mensaje por cliente, el sistema debe emitir del orden de **2.560 millones** de mensajes en dos horas. Ningún servidor hace eso, y además nadie necesita ese detalle: el usuario no distingue una actualización por acta de una actualización cada dos segundos. Debe **agregar por ventana de tiempo** y enviar un único mensaje con el cambio acumulado, calculando el delta mínimo que cada grupo de suscriptores necesita. Debe reportar cuántos mensajes emite su sistema en total y compararlo con la cifra ingenua.

RF-5. Suscripción por tópico

Un cliente público mira un municipio concreto o un puesto concreto, no todo el departamento. El fan-out debe entregar a cada cliente solo lo que le corresponde, con un costo proporcional al número de suscriptores de ese tópico y no al número total de conexiones de la instancia. Debe describir la estructura que mantiene esa relación entre tópicos y conexiones, y su costo de alta, baja y recorrido.

RF-6. Reconexión con recuperación de lo perdido

Un cliente que se desconecta treinta segundos debe poder volver y quedar **exactamente** en el mismo estado que uno que nunca se desconectó, sin descargar todo el acumulado nacional desde cero y sin perder las actualizaciones ocurridas durante su ausencia. Debe diseñar el mecanismo de continuidad: qué identificador lleva el cliente, qué le entrega el servidor al reconectar y en qué punto conviene mandar una fotografía completa en lugar del detalle de lo ocurrido.

> **Advertencia sobre el mecanismo de difusión**
> El mecanismo de publicación y suscripción clásico de Redis **no tiene memoria**: lo que se publica mientras un servidor está caído o un cliente desconectado se pierde para siempre. Si su diseño apoya la recuperación de RF-6 únicamente en ese mecanismo, no cumple el requisito. Debe sustentar qué estructura de Redis usa para conservar el historial reciente, cuánto historial retiene, con qué política de expiración y cuánta memoria le cuesta.

RF-7. Cola de verificación con asignación exclusiva

Las actas dudosas van a una cola ordenada por prioridad que los 120 auditores ven en vivo. Cuando un auditor toma un caso, ese caso debe quedar bloqueado para los demás de forma inmediata en **todas** las instancias, y la consola de los otros auditores debe reflejarlo sin recargar. Si el auditor cierra el navegador, pierde la conexión o simplemente se va a almorzar con el caso abierto, el caso debe volver a la cola de forma automática. Pero si el auditor sigue trabajando, su bloqueo no puede expirar y permitir que otro entre: debe resolver esa tensión y explicarla.

Además, la liberación de un bloqueo debe ser segura: un auditor cuyo bloqueo ya expiró y fue reasignado no puede, al terminar su trabajo tardío, liberar el bloqueo que ahora pertenece a otra persona. Debe explicar el mecanismo que lo impide.

RF-8. Presencia y reparto del trabajo

El coordinador de verificación ve quién está conectado, en qué está trabajando cada auditor y cuánto lleva. La presencia debe sobrevivir al hecho de que los auditores están repartidos entre las ocho instancias y debe **expirar sola**: una conexión que se corta sin aviso (un portátil que se queda sin batería) no puede dejar un auditor fantasma ocupando la lista para siempre.

RF-9. Clientes lentos y contrapresión

Una parte del público está en redes móviles saturadas. Si el servidor emite más rápido de lo que esos clientes consumen, el búfer de salida crece y la memoria del servidor se dispara hasta tumbarlo. Debe vigilar el volumen pendiente de envío de cada conexión y definir una política explícita: a partir de cierto umbral, dejar de enviar el detalle y mandar una fotografía del estado cuando el cliente se ponga al día, o desconectarlo con un código que le indique volver con continuidad. Debe demostrar la política con un cliente deliberadamente lento.

RF-10. Degradación y avalancha de reconexión

Dos escenarios obligatorios. Primero: si Redis deja de responder, el sistema no puede quedar en un estado silencioso donde los jurados creen que transmitieron y nadie recibió nada; defina y demuestre el comportamiento degradado. Segundo: cuando una instancia muere, sus 50.000 clientes reconectan de golpe contra las siete restantes; si todos reintentan al mismo tiempo, tumban lo que queda. Debe implementar la estrategia de reintento del cliente que evita esa avalancha y mostrar la gráfica de reconexiones por segundo con y sin ella.

5. Papel obligatorio de Redis

Redis no es un detalle de implementación de este caso: es el elemento que permite que ocho servidores que no comparten memoria se comporten como un solo sistema. Su solución debe usarlo, como mínimo, en los siguientes papeles, y el informe debe justificar la estructura elegida para cada uno.

Papel	Qué resuelve	Lo que se evalúa
Backplane de difusión	Que un acta recibida por la instancia 3 llegue a los clientes conectados a las otras siete	Diseño del espacio de canales: un canal global contra un canal por municipio o por tópico, y el costo de cada opción en tráfico entre instancias
Historial para recuperación	Que un cliente o un servidor que estuvo ausente recupere lo que se perdió (RF-6)	Estructura con historial, identificadores monótonos, ventana de retención, política de recorte y memoria consumida
Acumulados	Los totales por mesa, puesto, municipio y departamento, consultables y actualizables en caliente	Estructuras elegidas, costo de actualización y de lectura, y cómo se obtiene el ranking de candidatos sin recorrer todo
Operación atómica compuesta	Aplicar un acta, revertir una versión anterior y actualizar cuatro niveles como una sola operación indivisible (RF-3)	Uso de ejecución de scripts del lado del servidor, por qué garantiza atomicidad y qué restricciones impone
Idempotencia	Que un reenvío no sume dos veces (RF-1)	Registro de claves de idempotencia con expiración, y qué ocurre si la clave expira antes que el reintento
Bloqueo distribuido	La asignación exclusiva de un acta a un auditor (RF-7)	Bloqueo con dueño identificable y expiración, renovación mientras se trabaja y liberación segura verificando el dueño
Presencia	Quién está conectado y dónde, con expiración automática (RF-8)	Mecanismo de latido, tiempo de vida elegido y cómo se detectan las conexiones fantasma
Límite de tasa	Que un cliente mal configurado no inunde el sistema	Contador por ventana por credencial, y qué respuesta recibe quien excede el límite

6. Requerimientos técnicos obligatorios

Id	Exigencia	Cómo se verifica
RT-1	Mínimo **ocho instancias** de la aplicación detrás de un balanceador, desplegadas en la nube.	Se consulta el identificador de instancia desde varios clientes
RT-2	**Prohibidas las sesiones pegajosas**. El balanceador reparte sin afinidad y el sistema debe funcionar igual. Debe explicar qué problemas evita y cuáles le crea esa decisión.	Se abren varias conexiones y se verifica que caen en instancias distintas
RT-3	Ninguna instancia se comunica directamente con otra: toda la coordinación pasa por **Redis**.	Revisión de código y arquitectura
RT-4	El fan-out se hace por lotes cediendo el control al Event Loop. Debe instrumentar y mostrar el **retraso del Event Loop** de cada instancia durante el pico.	Panel de métricas durante la prueba de carga
RT-5	Las tres audiencias deben implementarse con el transporte que usted elija y justifique; al menos **dos transportes distintos** deben coexistir en el sistema entregado.	Revisión de código y sección 7 del informe

Id	Exigencia	Cómo se verifica
RT-6	La aplicación del jurado debe funcionar con conexión intermitente: encolar el acta y transmitirla cuando haya red, sin que el jurado tenga que recordar reenviarla.	Se corta la red durante la sustentación
RT-7	Instrumentación visible: conexiones por instancia, mensajes emitidos por segundo, retraso del Event Loop, memoria por conexión, actas por minuto y retraso entre la recepción de un acta y su publicación al público.	Tablero de operación durante la prueba
RT-8	Prueba de carga reproducible incluida en el repositorio, capaz de levantar al menos **20.000 conexiones simultáneas** contra el despliegue y de simular el perfil de la jornada.	Se ejecuta durante la sustentación
RT-9	La página pública debe seguir siendo usable en un teléfono de gama baja durante el pico: debe reportar el INP medido bajo carga.	Medición en vivo
RT-10	Despliegue con URL pública y funcionamiento verificable sin intervención del grupo.	Se abre la URL durante la sustentación